

Informe sobre Proyecto Final

Laura Valentina Giraldo Marín

Laura Isabella Patrouilleau Cañas

Ana Sofía Vásquez Castañeda

Universidad EAFIT

Escuela Ciencias Aplicadas e Ingeniería

Estructuras de Datos y Algoritmos

Germán Adolfo Montoya Orozco

Medellín, Colombia

Introducción

El proyecto consiste en el desarrollo de un editor de audio implementado en C++ completamente, cuyo objetivo principal es ofrecer herramientas básicas para procesar señales sonoras. El programa permite cargar archivos de audio, y aplicar distintas transformaciones por medio de librerías que permiten manejar los archivos de audio, procesar señales en el dominio de tiempo y frecuencia, y ver los resultados mediante una interfaz gráfica.

La lectura y escritura de archivos de audio se hace mediante libsndfile, que trabaja normalmente con formatos WAV, FLAC, AIFF. Para el análisis de espectro, se usó kissFFT, que implementa la Transformada Rápida de Fourier para representar la señal en el dominio de la frecuencia.

La parte gráfica del programa se construye con ImGui, que permite presentar controles, menús y visualizaciones. Finalmente, GLFW, se utiliza para crear la ventana del programa y renderizar las gráficas de señal de audio en tiempo real, permitiendo que la visualización sea más clara tanto en el tiempo como en la frecuencia.

El editor implementa funcionalidades básicas como: detección de picos en la señal, aumento o disminución del volumen, recorte o extracción de fragmentos de audio, visualización de la transformada FFT y una técnica de karaoke para cancelación de voces para archivos estéreo.

1. Funcionalidades de los directorios:

Libsndfile:

- Leer archivos WAV, FLAC y otros formatos
- Obtener información de la señal (frecuencia de muestreo, canales, frames)
- Extraer las muestras del audio como números (floats o ints)
- Guardar el audio procesado nuevamente en un archivo

kissFFT:

- Visualizar el espectro de frecuencias
- Identificar componentes dominantes
- Aplicar filtros y efectos
- Implementar técnicas como cancelación de voz o ecualización sencilla

ImGui:

- Botones (Cargar audio, Guardar, Recortar...)
- Sliders (ajustar volumen)
- Ventanas de visualización
- Gráficas y controles interactivos

GLFW:

- Crea la ventana del editor
- Maneja el input del usuario
- Conecta ImGui y OpenGL
- Controla el ciclo de renderizado

OpenGL: (Este directorio viene incluido por defecto en el equipo)

Permite visualizar:

- La forma de onda del audio
- El espectro de frecuencias (FFT)
- Barras, líneas, gráficos y cualquier representación visual

2. Instalación de directorios.

- **libsndfile:**

1. Se instala vcpkg: desde el PowerShell (normal, no administrador) se ejecutan los comandos uno por uno:

```
cd C:\
```

```
git clone https://github.com/microsoft/vcpkg.git
```

```
cd C:\vcpkg
```

Se ejecuta el instalador:

```
.\bootstrap-vcpkg.bat
```

Esto compila y activa vcpgk.

2. Instalar libsndfile: En PowerShell

```
.\vcpkg install libsndfile
```

Esto descargará, compilará e instalará libsndfile automáticamente. Cuando termine debería mostrar algo como:

```
libsndfile:x64-windows->installed successfully
```

3. Conectar vcpgk con CLion:

CLion usa CMake, entonces se debe especificar dónde están las librerías de vcpgk.

Se abre el archivo CMakeList.txt del proyecto, y se agrega antes del “add_executable(...)”:

find_package(SndFile CONFIG REQUIRED)

target_link_libraries(Prueba_audio PRIVATE SndFile::sndfile)

y se incluye: #include <sndfile.h> (cuando se vaya a usar en el código)

4. Indicarle a CLion dónde estopa vcpkg:

Para que CLion lo encuentre automáticamente se agrega esta variable:

En Clion:

File → Settings → Build, Execution, Deployment → CMake

En la sección CMake options, se agrega esta línea:

DCMAKE_TOOLCHAIN_FILE=C:/Users/lauis/vcpkg/scripts/buildsystems/vcpkg.c

make

(esto depende de donde esté la carpeta vcpgk)

Luego clic en Apply → OK y se vuelve a compilar el proyecto.

- **kissFFT:**

1. La descargamos desde el repositorio oficial (o un mirror estable):

<https://github.com/mborgerding/kissfft>

Luego se guardó dentro del proyecto, en una carpeta llamada:

Prueba_audio/kissfft/

Esa carpeta contenía los archivos:

kiss_fft.h

kiss_fft.c

kiss_fftr.h

kiss_fftr.c

2. Integración en CMake

En lugar de compilar KissFFT como una librería separada, lo incluimos directamente en el ejecutable.

En el CMakeLists.txt, agregamos sus fuentes en la lista del ejecutable:

```
add_executable(Prueba_audio)
```

```
    main.cpp
```

```
    Pista_audio.cpp
```

```
    Arbol_picos.cpp
```

```
    Procesador_audio.cpp
```

```
    Editor_logica.cpp
```

```
    mainwindow.cpp
```

```
# Archivos de KissFFT
```

```
kissfft/kiss_fft.c
```

```
kissfft/kiss_fftr.c
```

y luego le dijimos a CMake dónde encontrar los headers:

```
target_include_directories(Prueba_audio PRIVATE kissfft)
```

Esto permite que cualquier archivo del proyecto que tenga:

```
#include "kiss_fft.h"
```

```
#include "kiss_fftr.h"
```

pueda compilar sin problemas.

- **ImGui:**

1. Instalación de ImGui con vcpkg:

Desde PowerShell, ir a la carpeta de vcpkg:

```
cd C:\vcpkg
```

Instalar ImGui:

```
.\vcpkg install imgui
```

Instalar el backend OpenGL+GLFW:

```
.\vcpkg install imgui[glfw-binding,opengl3-binding]
```

Esto instala:

- ImGui base
- Binding para GLFW (input, ventana)
- Binding para OpenGL 3 (renderizado)

2. Integración CMake:

En CMakeList.txt, se agrega:

```
find_package(imgui CONFIG REQUIRED)  
target_link_libraries(tuProyecto PRIVATE imgui::imgui)
```

- **GLFW:**

1. Instalación de GLFW con vcpkg: En PowerShell:

```
cd C:\vcpkg  
.\vcpkg install glfw3
```

2. Integración con CMake:

```
find_package(glfw3 CONFIG REQUIRED)  
target_link_libraries(tuProyecto PRIVATE glfw)
```

3. Implementaciones y funcionalidades del código:

Main:

Este constituye el núcleo del programa, encargado de controlar la interacción con el usuario y coordinar las operaciones principales del editor de audio. Su función es servir como interfaz textual que permite ejecutar las opciones de procesamiento implementadas en la clase EditorLogica y visualizar resultados básicos como la FFT o la forma de onda:

```
#include <iostream>  
#include <string>  
#include <limits>  
#include <vector>  
#include <complex>
```

```
#include "Editor_logica.h"
#include "Visualizador_Audio.h"
```

Esta función imprime en pantalla el menú principal del editor, donde el usuario puede seleccionar una de las funcionalidades del editor

```
void mostrarMenu() {
    std::cout << "Selecciona una opcion:" << std::endl;
    std::cout << "1. Cargar archivo de audio" << std::endl;
    std::cout << "2. Aplicar volumen" << std::endl;
    std::cout << "3. Deshacer" << std::endl;
    std::cout << "4. Aplicar Karaoke" << std::endl;
    std::cout << "5. Encontrar Picos" << std::endl;
    std::cout << "6. Calcular FFT" << std::endl;
    std::cout << "7. Guardar archivo" << std::endl;
    std::cout << "8. Recortar Audio" << std::endl;
    std::cout << "9. Ver visualizacion de la senal" << std::endl;
    std::cout << "10. Salir" << std::endl;
    std::cout << "Opcion: ";
}
```

Esta función recibe los valores complejos de FFT y el sample rate. Y calcula: la magnitud de cada bin de frecuencia, y la frecuencia correspondiente a cada índice. Para no saturar la salida de la consola muestra solo un subconjunto de frecuencias. Esta permite visualizar de forma simple el contenido espectral del audio.

```
void imprimirResultadoFFT(const std::vector<std::complex<float>>& fftData, int sampleRate)
{
    std::cout << "Resultado de la FFT (Magnitud por Frecuencia):" << std::endl;
    size_t numBins = fftData.size();
    float freqPaso = (float)sampleRate / (2 * (numBins - 1));

    for (size_t i = 0; i < numBins; ++i) {
        float magnitud = std::abs(fftData[i]);
        float frecuencia = i * freqPaso;
        if (i % (numBins / 20) == 0) {
            std::cout << " Frecuencia: " << (int)frecuencia << " Hz, Magnitud: " << magnitud <<
std::endl;
        }
    }
}
```

En *main()* se crea un objeto "EditorLogica", que almacena y gestiona la pista de audio. El programa entra en un bucle (while) donde: muestra el menú, solicita una opción al usuario, y ejecuta la operación que corresponda. En cada caso se llama una función en "EditorLogica".


```

int main() {
    EditorLogica logica;

    std::cout << "Editor de Audio" << std::endl;

    bool salir = false;
    while (!salir) {
        mostrarMenu();
        int opcion;

        if (!(std::cin >> opcion)) {
            std::cout << "Error, ingresa un numero." << std::endl;
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            continue;
        }

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        std::string rutaEntrada;
        std::string rutaSalida;

        switch (opcion) {
            case 1:
                std::cout << "Ingresa el nombre del archivo a cargar: ";
                std::getline(std::cin, rutaEntrada);
                logica.cargarArchivo(rutaEntrada);
                break;

            case 2: {
                float factor;
                std::cout << "Ingresa el factor de volumen (ej: 1.5 para 150%, 0.5 para
50%)";
                std::cin >> factor;
                logica.aplicarVolumen(factor);
                break;
            }

            case 3:
                std::cout << "Aplicando Deshacer" << std::endl;
                logica.deshacer();
                break;

            case 4:
                std::cout << "Aplicando Efecto Karaoke" << std::endl;

```

```

        logica.aplicarKaraoke();
        break;
    case 5: {
        float umbral;
        std::cout << "Ingresa el minimo para buscar picos (ej: 0.2): ";
        std::cin >> umbral;
        logica.analizarPicos(umbral);
        break;
    }
    case 6: {
        std::cout << "Calculando FFT" << std::endl;
        auto resultadoFFT = logica.obtenerFFT(0);
        if (!resultadoFFT.empty()) {
            int sr = logica.getPistaActual().getSampleRate();
            imprimirResultadoFFT(resultadoFFT, sr);
        }
        break;
    }
    case 7:
        std::cout << "Ingresa el nombre para guardar el archivo";
        std::getline(std::cin, rutaSalida);
        logica.guardarEnArchivo(rutaSalida);
        break;

    case 8: {
        double inicio, fin;
        std::cout << "Recortar Audio" << std::endl;
        std::cout << "Ingresa el segundo inicial: ";
        std::cout << "Ingresa el segundo inicial: ";
        std::cin >> inicio;
        std::cout << "Ingresa el segundo de final: ";
        std::cin >> fin;
        logica.recortarAudio(inicio, fin);
        break;
    }

    case 9: {
        if (logica.getPistaActual().getNumFrames() == 0) {
            std::cout << "Debes cargar un archivo de audio antes de visualizar.";
            std::cout << std::endl;
        } else {
            VisualizadorAudio visualizador;
            visualizador.ejecutarGUI(logica);
        }
        break;
    }
    case 10:
        std::cout << "Fin del programa" << std::endl;

```

```

        salir = true;
        break;
    default:
        std::cout << "Opcion no valida. Intentalo de nuevo." << std::endl;
        break;
    }
}

return 0;
}

```

Opciones del menú:

1. En “Cargar Archivo” se lee la ruta que se ingresó, y envía la ruta a *logica.cargarArchivo()*. Permite abrir un archivo de audio para su posterior edición.
2. En “Aplicar Volumen” recibe un número y llama a *aplicarVolumen()*, aumentando o disminuyendo la amplitud de la señal.
3. En “Deshacer” llama a *logica.deshacer()*, que restaura la versión anterior de la señal.
4. En “Efecto Karaoke” ejecuta *logica.aplicarKaraoke()*, donde se realiza la técnica de cancelación de voz (eliminando la componente central compartida por los canales).
5. En “Detección de picos” solicita un umbral al usuario y luego llama a *analizarPicos()*. Sirve para identificar regiones donde la señal supera cierta amplitud.
6. En “Calcular FFT” llama a *logica.obtenerFFT()* y luego a *imprimirResultadoFFT()*. Permite inspeccionar el contenido en frecuencia del audio cargado.
7. En “Guardar Archivo” solicita una ruta y ejecuta *guardarEnArchivo()*, guardando la señal procesada.
8. En “Recortar audio” solicita el tiempo de inicio y final y llama a *recortarAudio()*. Genera una nueva señal que conserva únicamente ese rango temporal.
9. En “Visualización gráfica” si hay audio cargado, crea un objeto *VisualizadorAudio* y llama a *ejecutarGUI()*, abriendo la ventana gráfica generada con ImGui/GLFW/OpenGL.
10. En “Salir” termina el programa.

Árbol_picos.cpp:

Implementa una estructura de datos basada en un árbol binario de búsqueda que almacena los picos detectados en la señal de audio. Cada nodo del árbol representa un pico, definido por dos atributos: el tiempo en segundos donde ocurre y la amplitud del pico. Esta estructura permite organizar los picos de manera ordenada según su posición temporal y tiene métodos para insertarlos y mostrarlos.

```

#include "Arbol_picos.h"
#include <iostream>

```

```

ArbolPicos::ArbolPicos() : m_raiz(nullptr) {}

```

```
ArbolPicos::~~ArbolPicos() {
    destruirRecursivo(m_raiz);
}
```

Esta función inserta un nuevo pico en el árbol, manteniendo el orden por tiempo. Llama a la función recursiva para colocarlo en la posición correcta:

```
void ArbolPicos::insertar(double tiempo, float amplitud) {
    insertarRecursivo(m_raiz, tiempo, amplitud);
}
```

Esta función muestra en consola todos los picos almacenados. Si no hay ninguno, imprime un mensaje que dice que no se encontraron picos:

```
void ArbolPicos::imprimirPicos() const {
    if (m_raiz == nullptr) {
        std::cout << "No se encontraron picos." << std::endl;
    } else {
        std::cout << "Picos encontrados (Tiempo, Amplitud)" << std::endl;
        imprimirRecursivo(m_raiz);
    }
}
```

Esta función crea un nodo nuevo si el árbol está vacío, si el tiempo es menor, lo inserta en un subárbol a la izquierda, o si es mayor o igual, a la derecha:

```
void ArbolPicos::insertarRecursivo(NodoArbolPico*& nodo, double tiempo, float amplitud) {
    if (nodo == nullptr) {
        nodo = new NodoArbolPico(tiempo, amplitud);
    }
    else if (tiempo < nodo->tiempoSegundo) {
        insertarRecursivo(nodo->izquierdo, tiempo, amplitud);
    }
    else {
        insertarRecursivo(nodo->derecho, tiempo, amplitud);
    }
}
```

Esta función libera la memoria de los nodos del árbol mediante un recorrido (post-order):

```
void ArbolPicos::destruirRecursivo(NodoArbolPico* nodo) {
    if (nodo != nullptr) {
        destruirRecursivo(nodo->izquierdo);
        destruirRecursivo(nodo->derecho);
        delete nodo;
    }
}
```

Esta función recorre el árbol in-order, y muestra cada nodo garantizando que los picos se impriman ordenados por tiempo:

```
void ArbolPicos::imprimirRecursivo(NodoArbolPico* nodo) const {
    if (nodo != nullptr) {
        imprimirRecursivo(nodo->izquierdo);
        std::cout << " - " << nodo->tiempoSegundo << "s, " << nodo->amplitud << std::endl;
        imprimirRecursivo(nodo->derecho);
    }
}
```

Procesador_audio.cpp:

Procesa el dominio del tiempo y el análisis del espectro en el dominio de la frecuencia. Opera sobre los objetos de tipo *PistaAudio*, modificándolos o extrayendo información útil para etapas posteriores del código.

```
#include "Procesador_audio.h"
#include "kiss_fftr.h" // librería: kissfft
#include <cmath>
#include <iostream>
#include <vector>
```

Esta función aplica un escalado uniforme a todas las muestras de la pista, multiplicándolas por un factor determinado:

```
void ProcesadorAudio::aplicarVolumen(PistaAudio& pista, float factor) {
    if (pista.getNumFrames() == 0) return;
    for (auto& canal : pista.getCanalesEditables()) {
        for (float& muestra : canal) {
            muestra *= factor;
        }
    }
}
```

Esta función Implementa un filtrado estéreo para simular efecto “karaoke”. Cuando la pista tiene dos canales, resta el canal derecho del izquierdo para cancelar el contenido en la zona central del campo estéreo (donde suele ubicarse la voz). Produce una señal mono-replicada en ambos canales:

```
void ProcesadorAudio::aplicarKaraoke(PistaAudio& pista) {
    if (pista.getNumCanales() != 2) {
        std::cerr << "Karaoke solo funciona en pistas de dos canales." << std::endl;
        return;
    }
}
```

```

    auto& canales = pista.getCanalesEditables();
    for (long long i = 0; i < pista.getNumFrames(); ++i) {
float muestraMono = canales[0][i] - canales[1][i];
        canales[0][i] = muestraMono;
        canales[1][i] = muestraMono;
    }
}

```

Esta función genera un recorte temporal de la pista entre dos instantes dados. Convierte los tiempos a índices de muestra y crea nuevas señales por canal que contienen únicamente el rango solicitado:

```

void ProcesadorAudio::recortar(PistaAudio& pista, double tiempoInicio, double tiempoFin) {
    int sr = pista.getSampleRate();
    long long numFramesAntiguo = pista.getNumFrames();

    long long inicioSample = (long long)(tiempoInicio * sr);
    long long finSample = (long long)(tiempoFin * sr);

    if (inicioSample < 0) inicioSample = 0;
    if (finSample > numFramesAntiguo) finSample = numFramesAntiguo;

    if (inicioSample >= finSample || tiempoInicio >= tiempoFin) {
        std::cerr << "Rango de tiempo inválido." << std::endl;
        return;
    }

    std::vector<std::vector<float>> canalesNuevos;
    std::vector<std::vector<float>>& canalesAntiguos = pista.getCanalesEditables();

    for (const auto& canal : canalesAntiguos) {
        auto inicioIter = canal.begin() + inicioSample;
        auto finIter = canal.begin() + finSample;

        canalesNuevos.push_back(std::vector<float>(inicioIter, finIter));
    }

    canalesAntiguos = std::move(canalesNuevos);

    long long nuevosFrames = finSample - inicioSample;
    pista.setNumFrames(nuevosFrames);
}

```

Esta función calcula la Transformada Rápida de Fourier de un canal de audio usando la librería KISS FFT. Devuelve un vector de complejos que representa el espectro en frecuencia:

```

std::vector<std::complex<float>> ProcesadorAudio::calcularFFT(const std::vector<float>&
canal) {
    int nfft = canal.size();
    if (nfft == 0) {
        return {};
    }
    int nfft_out = (nfft / 2) + 1;

    kiss_fftr_cfg cfg = kiss_fftr_alloc(nfft, 0, nullptr, nullptr);
    if (cfg == nullptr) {
        std::cerr << "Error al configurar kissfft" << std::endl;
        return {};
    }

    std::vector<kiss_fft_cpx> espectro(nfft_out);
    kiss_fftr(cfg, canal.data(), espectro.data());
    kiss_fftr_free(cfg);

    std::vector<std::complex<float>> resultado;
    resultado.reserve(nfft_out);
    for (const auto& cpx : espectro) {
        resultado.push_back(std::complex<float>(cpx.r, cpx.i));
    }
    return resultado;
}

```

Esta función encuentra los picos y los almacena dentro del árbol binario de búsqueda:

```

ArbolPicos ProcesadorAudio::encontrarPicos(const PistaAudio& pista, float umbral) {
    ArbolPicos arbol;
    if (pista.getCanales().empty()) {
        return arbol;
    }
    const auto& canalCero = pista.getCanales()[0];
    for (long long i = 1; i < pista.getNumFrames() - 1; ++i) {
        float muestra = std::abs(canalCero[i]);
        if (muestra > umbral && muestra > std::abs(canalCero[i-1]) && muestra >=
std::abs(canalCero[i+1])) {double tiempo = (double)i / pista.getSampleRate();
        arbol.insertar(tiempo, muestra);
    }
    }
    return arbol;
}

```

Editor_logica.cpp:

Actúa como la capa de control y coordinación del editor de audio. Centraliza las operaciones realizadas sobre una pista, administra el sistema de undo, y conecta las acciones del usuario con las funciones de procesamiento implementadas en ProcesadorAudio.

Su responsabilidad principal es gestionar el estado de la pista cargada, ejecutar transformaciones, y permitir revertir cambios.

```
#include "Editor_logica.h"
#include "Procesador_audio.h"
#include <iostream>
#include <algorithm>
```

```
EditorLogica::EditorLogica() {}
```

Esta función guarda una copia completa del estado actual de la pista antes de aplicar cualquier modificación:

```
void EditorLogica::guardarEstadoParaUndo() {
    m_historialUndo.push(m_pistaActual);
}
```

Esta función carga un archivo de audio desde la ruta indicada. Luego, reinicia el historial de undo, borra cualquier análisis de picos previo, muestra información de la pista. Si hay alguna falla notifica el error:

```
void EditorLogica::cargarArchivo(const std::string& ruta) {
    if (m_pistaActual.cargarDesdeArchivo(ruta)) {
        m_historialUndo = std::stack<PistaAudio>();
        m_pistaActual.setPicosDetectados(ArbolPicos());
        std::cout << "Archivo cargado" << std::endl;
        std::cout << "Tasa de Muestreo: " << m_pistaActual.getSampleRate() << " Hz" <<
std::endl;
    } else {
        std::cerr << "No se pudo cargar el archivo." << std::endl;
    }
}
```

Esta función guarda la pista en un archive de salida. Si no funciona, notifica el error:


```

void EditorLogica::guardarEnArchivo(const std::string& ruta) {
    if (m_pistaActual.guardarEnArchivo(ruta)) {
        std::cout << "Archivo guardado en " << ruta << std::endl;
    } else {
        std::cerr << "No se pudo guardar el archivo." << std::endl;
    }
}

```

Esta función verifica que haya una pista cargada. Guarda el estado previo en el historial y aplica el cambio de volumen. Muestra un mensaje confirmando la operación:

```

void EditorLogica::aplicarVolumen(float factor) {
    if (m_pistaActual.getNumFrames() == 0) {
        std::cerr << "No hay archivo cargado." << std::endl;
        return;
    }
    guardarEstadoParaUndo();
    ProcesadorAudio::aplicarVolumen(m_pistaActual, factor);
    std::cout << "Volumen aplicado." << std::endl;
}

```

Esta función funciona igual que la acción anterior, pero ejecuta el algoritmo de cancelación central para producir un efecto tipo karaoke:

```

void EditorLogica::aplicarKaraoke() {
    if (m_pistaActual.getNumFrames() == 0) {
        std::cerr << "No hay archivo cargado." << std::endl;
        return;
    }
    guardarEstadoParaUndo();
    ProcesadorAudio::aplicarKaraoke(m_pistaActual);
    std::cout << "Efecto karaoke aplicado." << std::endl;
}

```

Esta función revierte la pista al último estado guardado en el historial. Si no existen estados previos, informa que no hay nada que deshacer:

```

void EditorLogica::deshacer() {
    if (!m_historialUndo.empty()) {
        m_pistaActual = m_historialUndo.top();
        m_historialUndo.pop();
        std::cout << "Operacion completada." << std::endl;
    }
}

```

```

    } else {
        std::cout << "No hay nada que deshacer." << std::endl;
    }
}

```

Esta función recorta la pista dentro del intervalo especificado. Antes de aplicar el recorte, guarda el estado actual para permitir un undo:

```

void EditorLogica::recortarAudio(double tiempoInicio, double tiempoFin) {
    if (m_pistaActual.getNumFrames() == 0) {
        std::cerr << "No hay archivo cargado." << std::endl;
        return;
    }
    guardarEstadoParaUndo();

    ProcesadorAudio::recortar(m_pistaActual, tiempoInicio, tiempoFin);

    std::cout << "Audio recortado." << std::endl;
}

```

Esta función ejecuta la detección de picos en la pista usando un umbral definido por el usuario. El resultado se almacena dentro de la propia pista. Luego imprime los picos encontrados:

```

void EditorLogica::analizarPicos(float umbral) {
    if (m_pistaActual.getNumFrames() == 0) {
        std::cerr << "No hay archivo cargado." << std::endl;
        return;
    }
    ArbolPicos nuevosPicos = ProcesadorAudio::encontrarPicos(m_pistaActual, umbral);
    m_pistaActual.setPicosDetectados(nuevosPicos);

    std::cout << "Análisis de picos completado." << std::endl;
    m_pistaActual.getPicosDetectados().imprimirPicos();
}

```

Esta función extrae un segmento de 4096 muestras del canal indicado y calcula su FFT, si el canal no tiene las muestras suficientes toma lo que esté disponible, completa con ceros y calcula igual el FFT y devuelve el espectro en el dominio de la frecuencia:

```

std::vector<std::complex<float>> EditorLogica::obtenerFFT(int canal) {
    if (m_pistaActual.getNumCanales() <= canal || m_pistaActual.getNumFrames() == 0) {

```

```

        return {};
    }

    const auto& datosCanal = m_pistaActual.getCanales()[canal];
    int sr = m_pistaActual.getSampleRate();

    size_t inicio = sr;
    size_t tamanoFFT = 4096;

    if (datosCanal.size() < inicio + tamanoFFT) {
        size_t tamanoReal = std::min((size_t)tamanoFFT, datosCanal.size());
        std::vector<float> segmento(datosCanal.begin(), datosCanal.begin() + tamanoReal);
        segmento.resize(tamanoFFT, 0.0f);
        return ProcesadorAudio::calcularFFT(segmento);
    }

    std::vector<float> segmento(datosCanal.begin() + inicio, datosCanal.begin() + inicio +
tamanoFFT);

    return ProcesadorAudio::calcularFFT(segmento);
}

```

Editor_logica.h:

Define la clase *EditorLogica*, el archivo *Editor_logica.h* define la clase *EditorLogica*, que funciona como el “cerebro” del editor de audio. Esta clase no procesa directamente las muestras, sino que coordina y controla todas las funciones principales del programa, utilizando la clase *PistaAudio* para almacenar el audio y la clase *ProcesadorAudio* para aplicar las operaciones.

```

#pragma once
#include "Pista_audio.h"
#include <stack>
#include <string>
#include <vector>
#include <complex>

class EditorLogica {
private:
    PistaAudio m_pistaActual;
    std::stack<PistaAudio> m_historialUndo;

    void guardarEstadoParaUndo();

public:
    EditorLogica();

```

```

void recortarAudio(double tiempoInicio, double tiempoFin);
void cargarArchivo(const std::string& ruta);
void guardarEnArchivo(const std::string& ruta);
void aplicarVolumen(float factor);
void aplicarKaraoke();
void deshacer();
void analizarPicos(float umbral);
std::vector<std::complex<float>> obtenerFFT(int canal);

const PistaAudio& getPistaActual() const { return m_pistaActual; }
};

```

Procesador_audio.h:

El archivo *Procesador_audio.h* declara la clase *ProcesadorAudio*, que reúne todas las operaciones técnicas de procesamiento directo sobre la señal de audio. A diferencia de *EditorLogica*, esta clase trabaja directamente sobre los datos de la pista, realizando modificaciones matemáticas sobre las muestras.

```

#pragma once
#include "Pista_audio.h"
#include "Arbol_picos.h"
#include <vector>
#include <complex>

class ProcesadorAudio {
public:
    static void aplicarVolumen(PistaAudio& pista, float factor);
    static void aplicarKaraoke(PistaAudio& pista);
    static void recortar(PistaAudio& pista, double tiempoInicio, double tiempoFin);

    static std::vector<std::complex<float>> calcularFFT(const std::vector<float>& canal);

    static ArbolPicos encontrarPicos(const PistaAudio& pista, float umbral);
};

```

Arbol_picos.h:

Define la clase *ArbolPicos*.

```

#pragma once
#include "Nodo_arbol_pico.h"

class ArbolPicos {
public:

```

```

ArbolPicos();
~ArbolPicos();

void insertar(double tiempo, float amplitud);

void imprimirPicos() const;

private:
    NodoArbolPico* m_raiz;

    void insertarRecursivo(NodoArbolPico*& nodo, double tiempo, float amplitud);
    void destruirRecursivo(NodoArbolPico* nodo);
    void imprimirRecursivo(NodoArbolPico* nodo) const;
};

```

Visualizador_audio.cpp:

El archivo Visualizador_Audio.cpp implementa la clase encargada de toda la interfaz gráfica del editor mediante GLFW, OpenGL, ImGui e ImPlot. Su función es crear la ventana del programa, gestionar el ciclo de renderizado y mostrar visualmente la información de la pista de audio. El visualizador permite graficar la forma de onda en el dominio del tiempo y la FFT en el dominio de la frecuencia, obteniendo los datos desde la lógica del editor. Además, ofrece controles interactivos para seleccionar el canal, actualizar las gráficas y analizar la señal de forma intuitiva. En conjunto, este módulo actúa como la parte visual del sistema, permitiendo inspeccionar el audio de manera clara y en tiempo real

```

#include "Visualizador_Audio.h"
#include "implot.h"
#include "imgui.h"
#include <cmath>
#include <iostream>
#include <GLFW/glfw3.h>
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"

VisualizadorAudio::VisualizadorAudio() :
    m_window(nullptr), // Inicializar el puntero a nullptr
    m_canalSeleccionado(0),
    m_factorVolumen(1.0f),
    m_umbralPicos(0.5f)
{
    m_rutaArchivo[0] = '\0';
}

bool VisualizadorAudio::inicializarGUI() {
    if (!glfwInit()) {

```

```

        std::cerr << "Error al inicializar GLFW.\n";
        return false;
    }
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

    m_window = glfwCreateWindow(SCREEN_WIDTH, SCREEN_HEIGHT, "Visualizador de
Audio", NULL, NULL);
    if (m_window == NULL) {
        glfwTerminate();
        return false;
    }
    glfwMakeContextCurrent(m_window);
    glfwSwapInterval(1);
    ImGui_CHECKVERSION();
    ImGui::CreateContext();
    ImPlot::CreateContext();
    ImGuiIO& io = ImGui::GetIO(); (void)io;
    ImGui::StyleColorsDark();

    ImGui_ImplGlfw_InitForOpenGL(m_window, true);
    ImGui_ImplOpenGL3_Init("#version 330");

    return true;
}

void VisualizadorAudio::limpiarGUI() {
    ImPlot::DestroyContext();
    ImGui_ImplOpenGL3_Shutdown();
    ImGui_ImplGlfw_Shutdown();
    ImGui::DestroyContext();

    if (m_window) {
        glfwDestroyWindow(m_window);
    }
    glfwTerminate();
}

void VisualizadorAudio::ejecutarGUI(EditorLogica& logica) {
    if (!inicializarGUI()) {
        return;
    }

    while (!glfwWindowShouldClose(m_window)) {
        glfwPollEvents();

        // 1. Iniciar el frame de ImGui
        ImGui_ImplOpenGL3_NewFrame();

```

```

ImGui_ImplGlfw_NewFrame();
ImGui::NewFrame();

DibujarPaneles(logica);

ImGui::Render();
int display_w, display_h;
glfwGetFramebufferSize(m_window, &display_w, &display_h);
glViewport(0, 0, display_w, display_h);
glClearColor(0.2f, 0.2f, 0.2f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);

ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

glfwSwapBuffers(m_window);
}

limpiarGUI();
std::cout << "Se cerro la ventana de visualizacion.\n";
}
void VisualizadorAudio::DibujarPaneles(EditorLogica& logica) {
    ImGui::Begin("Panel de Visualizacion de Audio");

    const PistaAudio& pista = logica.getPistaActual();

    if (pista.getNumFrames() > 0) {
        // Selector de canal
        if (pista.getNumCanales() > 1) {
            ImGui::Text("Seleccionar Canal:");
            ImGui::RadioButton("Izquierdo (0)", &m_canalSeleccionado, 0);
            ImGui::SameLine();
            ImGui::RadioButton("Derecho (1)", &m_canalSeleccionado, 1);
        } else {
            ImGui::Text("Pista Mono (Canal 0)");
            m_canalSeleccionado = 0;
        }
    }

    // 1. Prepara los datos (Tiempo y FFT)
    actualizarDatosGraficos(pista, m_canalSeleccionado, logica);

    // 2. Dibuja las gráficas
    dibujarGraficoTiempo(pista, m_canalSeleccionado);
    ImGui::Separator();
    dibujarGraficoFrecuencia(pista.getSampleRate());

} else {
    ImGui::Text("Carga un archivo para visualizar las gráficas.");
}
}

```

```

    ImGui::End();
}

void VisualizadorAudio::actualizarDatosGraficos(const PistaAudio& pista, int canal,
EditorLogica& logica) {
    if (pista.getNumFrames() == 0 || pista.getNumCanales() <= canal) {
        m_datosTiempoY.clear();
        m_datosTiempoX.clear();
        m_datosFrecuenciaMagnitud.clear();
        m_datosFrecuenciaX.clear();
        return;
    }
    const auto& datosCanal = pista.getCanales()[canal];
    long long numFrames = pista.getNumFrames();
    int sampleRate = pista.getSampleRate();

    size_t segmentoFrames = (size_t)numFrames;

    m_datosTiempoY.resize(segmentoFrames);
    m_datosTiempoX.resize(segmentoFrames);

    for (size_t i = 0; i < segmentoFrames; ++i) {
        m_datosTiempoY[i] = datosCanal[i];
        m_datosTiempoX[i] = (float)i / sampleRate;
    }

    if (m_datosFrecuenciaMagnitud.empty() || ImGui::IsItemDeactivatedAfterEdit() ||
ImGui::Button("Recalcular FFT")) {
        auto resultadoFFT = logica.obtenerFFT(canal);
        if (!resultadoFFT.empty()) {
            size_t numBins = resultadoFFT.size();
            m_datosFrecuenciaMagnitud.resize(numBins);
            m_datosFrecuenciaX.resize(numBins);

            float freqPaso = (float)sampleRate / (float)(2 * (numBins - 1));

            for (size_t i = 0; i < numBins; ++i) {
                m_datosFrecuenciaMagnitud[i] = std::abs(resultadoFFT[i]);
                m_datosFrecuenciaX[i] = i * freqPaso;
            }
        }
    }
}

void VisualizadorAudio::dibujarGraficoTiempo(const PistaAudio& pista, int canal) {
    if (m_datosTiempoY.empty()) return;

    std::string titulo = "1. Dominio del Tiempo (Amplitud) | Canal " + std::to_string(canal);

```



```

    if (ImPlot::BeginPlot(titulo.c_str(), ImVec2(-1, 300))) {

        ImPlot::SetupAxes("Tiempo (s)", "Amplitud Normalizada", ImPlotAxisFlags_None,
            ImPlotAxisFlags_RangeFit);
        ImPlot::SetupAxesLimits(0, m_datosTiempoX.back(), -1.0, 1.0, ImPlotCond_Always);

        // Dibuja la señal de audio
        ImPlot::PlotLine("Forma de Onda",
            m_datosTiempoX.data(),
            m_datosTiempoY.data(),
            m_datosTiempoY.size());

        ImPlot::EndPlot();
    }
}

void VisualizadorAudio::dibujarGraficoFrecuencia(int sampleRate) {
    if (m_datosFrecuenciaMagnitud.empty()) return;

    std::string titulo = "2. Dominio de la Frecuencia (FFT) | Max Freq: " +
        std::to_string(sampleRate / 2) + " Hz";

    if (ImPlot::BeginPlot(titulo.c_str(), ImVec2(-1, 300))) {

        ImPlot::SetupAxes("Frecuencia (Hz)", "Magnitud", ImPlotAxisFlags_AutoFit,
            ImPlotAxisFlags_RangeFit);

        ImPlot::PlotStems("Espectro",
            m_datosFrecuenciaX.data(),
            m_datosFrecuenciaMagnitud.data(),
            m_datosFrecuenciaMagnitud.size());

        ImPlot::EndPlot();
    }
}

```

Visualizador_audio.h:

Esta sección del código define las variables y métodos que el visualizador usa para mostrar la señal de audio tanto en el dominio del tiempo como en el de la frecuencia. Se almacenan arreglos con los datos necesarios para dibujar las gráficas (valores de amplitud, tiempo y magnitud de la FFT), así como información del estado actual del editor, como el canal seleccionado, el factor de volumen, la ruta del archivo cargado y el umbral para la detección de picos. Las funciones asociadas se encargan de graficar la forma de onda, representar el espectro de frecuencias y actualizar los datos visuales procesados por la lógica del editor, permitiendo que la interfaz refleje en tiempo real las modificaciones aplicadas al audio.

```

#pragma once
#include "Editor_logica.h"
#include <vector>

struct GLFWwindow;

class VisualizadorAudio {
public:
    VisualizadorAudio();

    void ejecutarGUI(EditorLogica& logica);

private:
    GLFWwindow* m_window;
    const int SCREEN_WIDTH = 1280;
    const int SCREEN_HEIGHT = 720;

    bool inicializarGUI();
    void limpiarGUI();

    void DibujarPaneles(EditorLogica& logica);
    std::vector<float> m_datosTiempoY;
    std::vector<float> m_datosTiempoX;
    std::vector<float> m_datosFrecuenciaMagnitud;
    std::vector<float> m_datosFrecuenciaX;

    int m_canalSeleccionado;
    float m_factorVolumen;
    char m_rutaArchivo[256];
    float m_umbralPicos;

    void dibujarGraficoTiempo(const PistaAudio& pista, int canal);
    void dibujarGraficoFrecuencia(int sampleRate);
    void actualizarDatosGraficos(const PistaAudio& pista, int canal, EditorLogica& logica);
};

```

Pista_audio.cpp

El código implementa una clase encargada de representar y gestionar una pista de audio. Permite cargar archivos de sonido desde disco usando la librería libsndfile, separando los datos en canales independientes para su procesamiento. También permite guardar nuevamente la pista en un archivo WAV. La clase almacena información esencial del audio, como la tasa de muestreo, la cantidad de frames y los datos de cada canal. Además, incluye soporte para guardar los picos detectados en el audio mediante un árbol.

```

#include "Pista_audio.h"
#include <sndfile.h> // librería libsndfile
#include <iostream>
#include <vector>

PistaAudio::PistaAudio() : m_sampleRate(0), m_numFrames(0) {}
void PistaAudio::setPicosDetectados(const ArbolPicos& picos) {
    m_picosDetectados = picos;
}

```

Esta función guarda en la pista el árbol de picos detectados:

```

bool PistaAudio::cargarDesdeArchivo(const std::string& ruta) {
    SF_INFO sflInfo;
    SNDFILE* archivo = sf_open(ruta.c_str(), SFM_READ, &sflInfo);
    if (!archivo) {
        std::cerr << "Error al abrir el archivo" << ruta << std::endl;
        return false;
    }
    this->m_sampleRate = sflInfo.samplerate;
    this->m_numFrames = sflInfo.frames;
    int numCanales = sflInfo.channels;

    std::vector<float> bufferIntercalado(m_numFrames * numCanales);
    sf_readf_float(archivo, bufferIntercalado.data(), m_numFrames);

    m_canales.clear();
    m_canales.resize(numCanales);
    for (int c = 0; c < numCanales; ++c) {
        m_canales[c].resize(m_numFrames);
    }

    for (long long i = 0; i < m_numFrames; ++i) {
        for (int c = 0; c < numCanales; ++c) {
            m_canales[c][i] = bufferIntercalado[i * numCanales + c];
        }
    }

    sf_close(archivo);
    return true;
}

```

Esta función escribe la pista actual en un archivo WAV y recombina los canales en un buffer, crea un archivo WAV con las propiedades correspondientes, y escribe los datos usando libsndfile y cierra el archivo.

```

bool PistaAudio::guardarEnArchivo(const std::string& ruta) {
    if (m_canales.empty() || m_numFrames == 0) {
        std::cerr << "No hay datos de audio para guardar." << std::endl;
        return false;
    }

    int numCanales = m_canales.size();

    SF_INFO sflInfo;
    sflInfo.frames = m_numFrames;
    sflInfo.samplerate = m_sampleRate;
    sflInfo.channels = numCanales;
    sflInfo.format = SF_FORMAT_WAV | SF_FORMAT_PCM_16; // WAV 16-bit

    SNDFILE* archivo = sf_open(ruta.c_str(), SFM_WRITE, &sflInfo);
    if (!archivo) {
        std::cerr << "Error al abrir el archivo" << ruta << std::endl;
        return false;
    }

    std::vector<float> bufferIntercalado(m_numFrames * numCanales);
    for (long long i = 0; i < m_numFrames; ++i) {
        for (int c = 0; c < numCanales; ++c) {
            bufferIntercalado[i * numCanales + c] = m_canales[c][i];
        }
    }

    sf_writeln_float(archivo, bufferIntercalado.data(), m_numFrames);

    sf_close(archivo);
    return true;
}

```

Pista_audio.h:

Funciona como el contenedor principal de los datos de una pista de audio dentro del programa. Almacena los canales del sonido, la tasa de muestreo y el número total de frames, y permite cargar o guardar archivos utilizando la librería libsndfile. También mantiene un árbol con los picos detectados en la señal, útil para análisis posteriores. La clase ofrece acceso tanto de lectura como de edición a los datos de los canales, y define constructores y operador de asignación para copiar pistas correctamente sin duplicar estructuras pesadas como el árbol de picos.

```

#pragma once
#include <string>
#include <vector>
#include "Arbol_picos.h"

```

```

class PistaAudio {
private:
    std::vector<std::vector<float>> m_canales;
    int m_sampleRate;
    long long m_numFrames;
    ArbolPicos m_picosDetectados;

public:
    PistaAudio();

    bool cargarDesdeArchivo(const std::string& ruta);
    bool guardarEnArchivo(const std::string& ruta);
    int getSampleRate() const { return m_sampleRate; }
    long long getNumFrames() const { return m_numFrames; }
    int getNumCanales() const { return m_canales.size(); }
    const std::vector<std::vector<float>>& getCanales() const { return m_canales; }

    std::vector<std::vector<float>>& getCanalesEditables() { return m_canales; }

    const ArbolPicos& getPicosDetectados() const { return m_picosDetectados; }

    void setPicosDetectados(const ArbolPicos&);

    void setNumFrames(long long newNumFrames) { m_numFrames = newNumFrames; }

    PistaAudio& operator=(const PistaAudio& otra) {
        if (this != &otra) {
            m_canales = otra.m_canales;
            m_sampleRate = otra.m_sampleRate;
            m_numFrames = otra.m_numFrames;
            m_picosDetectados = ArbolPicos();
        }
        return *this;
    }
    PistaAudio(const PistaAudio& otra) {
        m_canales = otra.m_canales;
        m_sampleRate = otra.m_sampleRate;
        m_numFrames = otra.m_numFrames;
        m_picosDetectados = ArbolPicos();
    }
};

```

Nodo_arbol_pico.h:

Representa un nodo dentro del árbol binario usado para almacenar picos detectados en una señal de audio. Cada nodo guarda el tiempo exacto del pico y su amplitud, junto con punteros a sus nodos hijo izquierdo y derecho. Esta estructura es la base del árbol de picos, permitiendo organizar los valores de manera jerárquica para búsquedas, inserciones y recorridos eficientes durante el análisis de la señal.

```
#pragma once
```

```
struct NodoArbolPico {  
    double tiempoSegundo;  
    float amplitud;  
  
    NodoArbolPico* izquierdo;  
    NodoArbolPico* derecho;  
  
    NodoArbolPico(double tiempo, float amp)  
        : tiempoSegundo(tiempo), amplitud(amp), izquierdo(nullptr), derecho(nullptr) {}  
};
```

CMAKELIST:

Se especifican todos los archivos fuente que componen el programa, incluyendo la lógica del editor, el procesamiento de audio, la visualización y las librerías externas como ImGui, ImPlot y KissFFT. También se añaden rutas de inclusión necesarias para acceder a los encabezados de estas librerías. Finalmente, se enlazan las dependencias requeridas para la compilación: GLFW, OpenGL y libsndfile, asegurando que el proyecto pueda manejar gráficos, ventanas y lecturas.

```
cmake_minimum_required(VERSION 3.20)  
project(AudioVisualizador)  
  
set(CMAKE_CXX_STANDARD 17)  
set(IMGUI_SOURCES  
    libs/imgui-master/imgui-master/imgui.cpp  
    libs/imgui-master/imgui-master/imgui_draw.cpp  
    libs/imgui-master/imgui-master/imgui_widgets.cpp  
    libs/imgui-master/imgui-master/imgui_tables.cpp  
    libs/imgui-master/imgui-master/imgui_impl_glfw.cpp  
    libs/imgui-master/imgui-master/imgui_impl_opengl3.cpp  
)  
set(IMPLOT_SOURCES  
    libs/implot-master/implot-master/implot.cpp  
    libs/implot-master/implot-master/implot_items.cpp  
)  
  
find_package(SndFile CONFIG REQUIRED)
```

```

find_package(glfw3 REQUIRED)
find_package(OpenGL REQUIRED)
add_executable(Prueba_audio
    main.cpp
    ${IMGUI_SOURCES}
    ${IMPLICIT_SOURCES}
    Pista_audio.cpp
    Arbol_picos.cpp
    Procesador_audio.cpp
    Editor_logica.cpp
    kissfft/kiss_fft.c
    kissfft/kiss_fftr.c
    Visualizador_audio.cpp
)

target_include_directories(Prueba_audio PRIVATE
    libs/imgui-master/imgui-master
    libs/implot-master/implot-master
    kissfft)

target_link_libraries(Prueba_audio PRIVATE
    glfw
    OpenGL::GL
    SndFile::sndfile
)

```

4. Resultados:

```
Editor de Audio
Selecciona una opcion:
1. Cargar archivo de audio
2. Aplicar volumen
3. Deshacer
4. Aplicar Karaoke
5. Encontrar Picos
6. Calcular FFT
7. Guardar archivo
8. Recortar Audio
9. Ver visualizacion de la senal
10. Salir
Opcion:1

Ingresa el nombre del archivo a cargar:Louis.wav

Archivo cargado
Tasa de Muestreo: 44100 Hz
```

Se muestra el menú al correr el programa y se carga un archivo de audio en formato WAV que se encuentre en la carpeta *cmake-build-debug*, luego muestra la tasa de muestreo que corresponde a 1 segundo para que en el FTT haya buena información.

```
Selecciona una opcion:
1. Cargar archivo de audio
2. Aplicar volumen
3. Deshacer
4. Aplicar Karaoke
5. Encontrar Picos
6. Calcular FFT
7. Guardar archivo
8. Recortar Audio
9. Ver visualizacion de la senal
10. Salir
Opcion:5

Ingresa el minimo para buscar picos (ej: 0.2):0.18
```

Se selecciona la opción de mostrar picos, se elige un número del cuál se buscarán amplitudes mayores a este.


```
Analisis de picos completado:
Picos encontrados (Tiempo, Amplitud)
- 6.5932s, 0.190796
- 6.59338s, 0.180511
- 6.59345s, 0.189209
- 6.59458s, 0.19989
- 6.59796s, 0.182098
- 7.3985s, 0.190308
- 7.39882s, 0.197388
- 7.75889s, 0.181396
- 7.78805s, 0.183105
- 9.00934s, 0.181641
- 9.79082s, 0.19751
- 9.79091s, 0.19693
- 9.791s, 0.198364
- 10.2832s, 0.181488
- 10.6113s, 0.193237
- 10.9864s, 0.185913
- 10.9924s, 0.189484
- 10.9925s, 0.186737
- 11.2458s, 0.184235
- 11.3955s, 0.182251
- 11.3957s, 0.193359
- 11.3959s, 0.183685
```

Este es el resultado de los primeros segundos del audio donde se muestra a la izquierda el segundo en el que se encontró el pico y a la derecha la magnitud de la amplitud.

```
- 187.013s, 0.196594
- 187.053s, 0.183807
- 187.088s, 0.199982
- 187.088s, 0.180725
- 187.104s, 0.184845
- 187.116s, 0.188416
- 187.152s, 0.186218
- 187.152s, 0.191711
- 187.156s, 0.192169
- 187.186s, 0.183289
- 187.187s, 0.198578
- 187.189s, 0.19989
- 187.19s, 0.196655
- 187.19s, 0.183624
- 187.192s, 0.189514
- 187.192s, 0.199982
- 187.257s, 0.184113
- 187.274s, 0.184113
- 187.334s, 0.18692
```

Este es el resultado de los últimos segundos del audio donde se muestra a la izquierda el segundo en el que se encontró el pico y a la derecha la magnitud de la amplitud.

```
Selecciona una opcion:
1. Cargar archivo de audio
2. Aplicar volumen
3. Deshacer
4. Aplicar Karaoke
5. Encontrar Picos
6. Calcular FFT
7. Guardar archivo
8. Recortar Audio
9. Ver visualizacion de la senal
10. Salir
Opcion:6
```

```
Calculando FFT
```

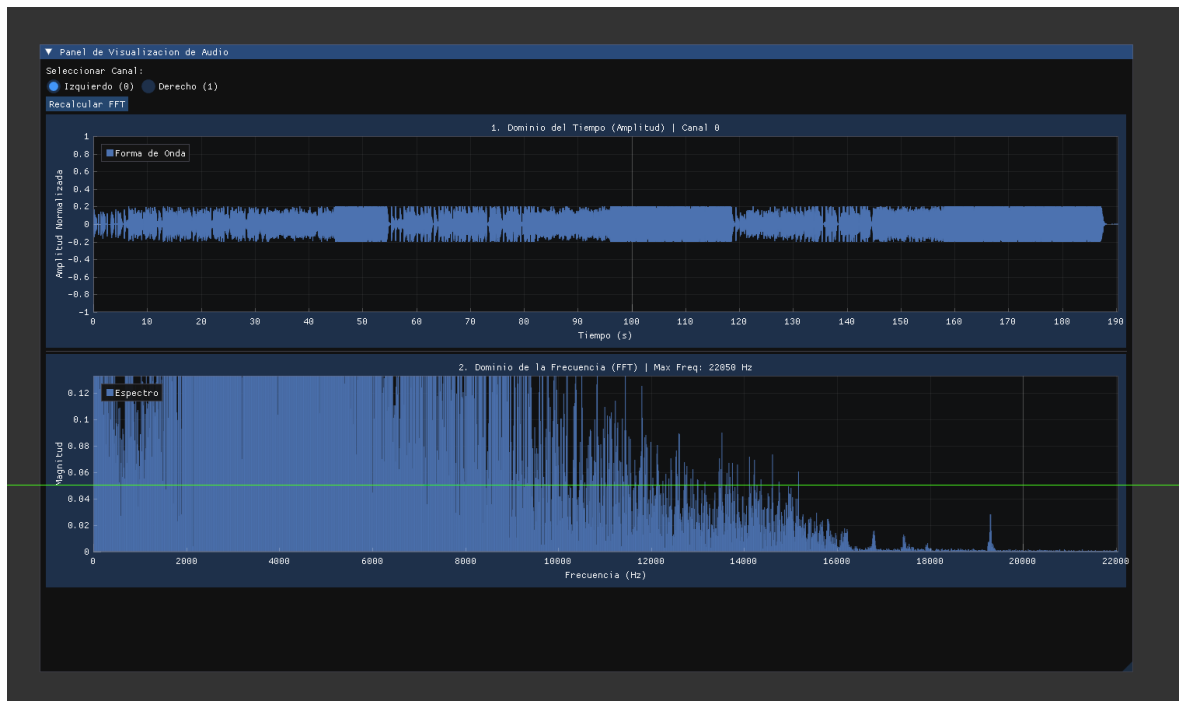
```
Resultado de la FFT (Magnitud por Frecuencia):
```

```
Frecuencia: 0 Hz, Magnitud: 0.132843
Frecuencia: 1098 Hz, Magnitud: 0.0550588
Frecuencia: 2196 Hz, Magnitud: 0.492411
Frecuencia: 3294 Hz, Magnitud: 1.77003
Frecuencia: 4392 Hz, Magnitud: 0.565584
Frecuencia: 5490 Hz, Magnitud: 0.80476
Frecuencia: 6589 Hz, Magnitud: 0.0780669
Frecuencia: 7687 Hz, Magnitud: 0.151948
Frecuencia: 8785 Hz, Magnitud: 0.0861594
Frecuencia: 9883 Hz, Magnitud: 0.0504483
```

Se selecciona la opción número 6 que corresponde a la muestra del FTT, en el lado izquierdo se muestra la frecuencia de los primeros segundos y en el derecho la magnitud de la amplitud en ese tiempo.

```
Selecciona una opcion:
1. Cargar archivo de audio
2. Aplicar volumen
3. Deshacer
4. Aplicar Karaoke
5. Encontrar Picos
6. Calcular FFT
7. Guardar archivo
8. Recortar Audio
9. Ver visualizacion de la senal
10. Salir
Opcion:9
```

Se selecciona la opción 9 que corresponde a la visualización de la señal.



Se abre una ventana creada por glfw donde se muestra de forma visual la señal en el tiempo y la señal de la frecuencia.

```
Selecciona una opcion:  
1. Cargar archivo de audio  
2. Aplicar volumen  
3. Deshacer  
4. Aplicar Karaoke  
5. Encontrar Picos  
6. Calcular FFT  
7. Guardar archivo  
8. Recortar Audio  
9. Ver visualizacion de la senal  
10. Salir  
Opcion:2
```

Ingresa el factor de volumen (ej: 1.5 para 150%, 0.5 para 50%)1.5

Volumen aplicado.

Seleccionamos la opción dos para aplicar un porcentaje de volumen representado de forma decimal.

```
Selecciona una opcion:
1. Cargar archivo de audio
2. Aplicar volumen
3. Deshacer
4. Aplicar Karaoke
5. Encontrar Picos
6. Calcular FFT
7. Guardar archivo
8. Recortar Audio
9. Ver visualizacion de la senal
10. Salir
Opcion:4
```

```
Aplicando Efecto Karaoke
Efecto karaoke aplicado.
```

Con la opción 4 se aplica el efecto karaoke donde se restan los canales para que se cancele la voz.

```
Selecciona una opcion:
1. Cargar archivo de audio
2. Aplicar volumen
3. Deshacer
4. Aplicar Karaoke
5. Encontrar Picos
6. Calcular FFT
7. Guardar archivo
8. Recortar Audio
9. Ver visualizacion de la senal
10. Salir
Opcion:8
```

```
Recortar Audio
Ingresa el segundo inicial:10

Ingresa el segundo de final:60

Audio recortado.
```

Al elegir la opción número 8 del menú el usuario puede elegir los segundos donde quiere recortar la canción y al final ese fragmento es el que queda.

```
Selecciona una opcion:
1. Cargar archivo de audio
2. Aplicar volumen
3. Deshacer
4. Aplicar Karaoke
5. Encontrar Picos
6. Calcular FFT
7. Guardar archivo
8. Recortar Audio
9. Ver visualizacion de la senal
10. Salir
Opcion:7

Ingresa el nombre para guardar el archivoAudio_volumen_recorte_karaoke.wav

Archivo guardado en Audio_volumen_recorte_karaoke.wav
```

Al guardar el archivo se debe elegir un nombre WAV y este se guarda como una copia modificada en la misma carpeta donde estaba el archivo original.

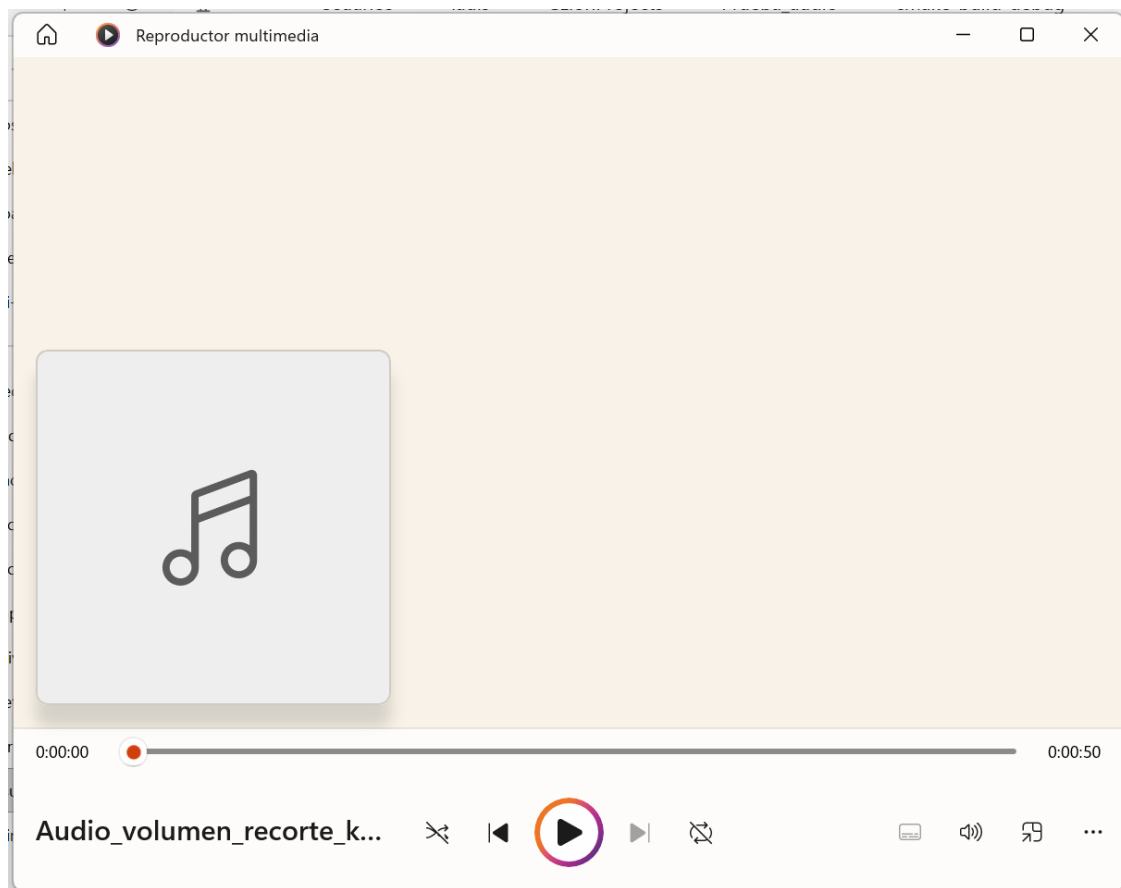
```
Selecciona una opcion:
1. Cargar archivo de audio
2. Aplicar volumen
3. Deshacer
4. Aplicar Karaoke
5. Encontrar Picos
6. Calcular FFT
7. Guardar archivo
8. Recortar Audio
9. Ver visualizacion de la senal
10. Salir
Opcion:
10

Fin del programa
```

Con la opción 10 se sale del editor.

🖥️ > ... Usuarios > lauis > CLionProjects > Prueba_audio > cmake-build-debug >				
📁 📄 🔗 🗑️ ⬆️ Ordenar ▾ ☰ Ver ▾ ⋮				
Nombre	Fecha de modificación	Tipo	Tamaño	
📁 .cmake	12/11/2025 9:11 p. m.	Carpeta de archivos		
📁 CMakeFiles	13/11/2025 9:13 p. m.	Carpeta de archivos		
📁 Testing	12/11/2025 9:11 p. m.	Carpeta de archivos		
📄 .gitignore	12/11/2025 9:11 p. m.	txtfile	1 KB	
📄 .ninja_deps	13/11/2025 10:08 p. m.	Archivo NINJA_DEPS	89 KB	
📄 .ninja_log	13/11/2025 10:08 p. m.	Archivo NINJA_LOG	4 KB	
🔊 Audio_volumen_recorte_karaoke	14/11/2025 6:26 p. m.	Archivo WAV	8.614 KB	
📄 build.ninja	13/11/2025 9:13 p. m.	Archivo NINJA	31 KB	
📄 cmake_install.cmake	12/11/2025 9:11 p. m.	Archivo CMAKE	3 KB	
📄 CMakeCache	13/11/2025 9:13 p. m.	Documento de texto	21 KB	
🔊 FLAC.dll	11/11/2025 10:03 p. m.	Extensión de la aplica...	778 KB	
🔊 glfw3.dll	13/11/2025 3:06 p. m.	Extensión de la aplica...	471 KB	
📄 imgui	14/11/2025 6:24 p. m.	Opciones de configu...	1 KB	
🔊 libgcc_s_seh-1.dll	20/06/2025 5:32 a. m.	Extensión de la aplica...	729 KB	
🔊 libmp3lame.DLL	9/10/2025 7:07 p. m.	Extensión de la aplica...	508 KB	
🔊 libwinpthread-1.dll	20/06/2025 5:32 a. m.	Extensión de la aplica...	69 KB	
🔊 Louis	12/11/2025 10:04 p. m.	Archivo WAV	32.810 KB	
🔊 mpg123.dll	9/10/2025 7:08 p. m.	Extensión de la aplica...	606 KB	
🔊 ogg.dll	11/11/2025 10:03 p. m.	Extensión de la aplica...	88 KB	
🔊 opus.dll	11/11/2025 10:03 p. m.	Extensión de la aplica...	894 KB	

Acá podemos ver la carpeta *cmake-build-debug* donde están los dos archivos, el inicial *Louis.wav* y el modificado *Audio_volumen_recorte_karaoke.wav*.



Muestra de la nueva duración del audio recortado.